

香港中文大學

The Chinese University of Hong Kong



DEPARTMENT OF  
EARTH AND  
ENVIRONMENTAL  
SCIENCES  
地球與環境科學系



Tai Group for  
Atmosphere-Biosphere  
Interactions

# Hybrid Modeling for Air Pollution Simulation: A Practical Example to Understand ML Fundamentals, Modeling Workflow, and a Quick-Start Guide

**MAO Jia**

Postdoctoral Fellow

Department of Earth and Environmental Sciences

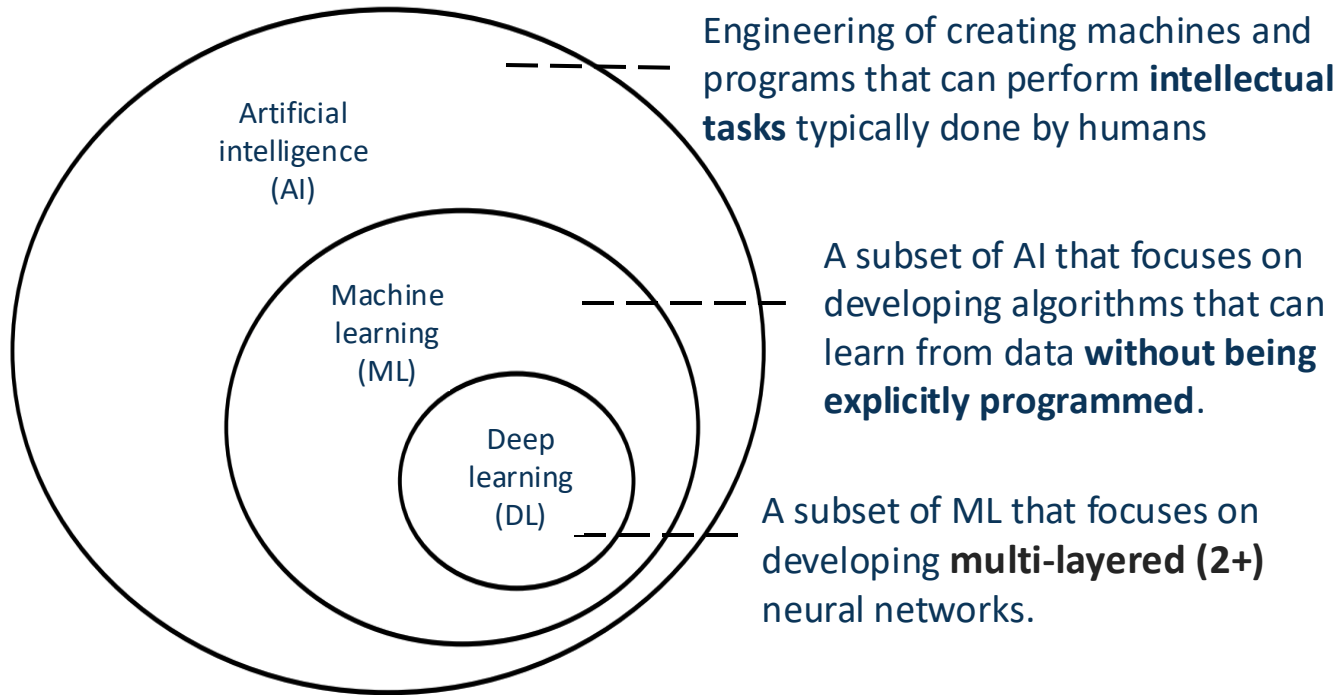
The Chinese University of Hong Kong

Email: [maojia@link.cuhk.edu.hk](mailto:maojia@link.cuhk.edu.hk)

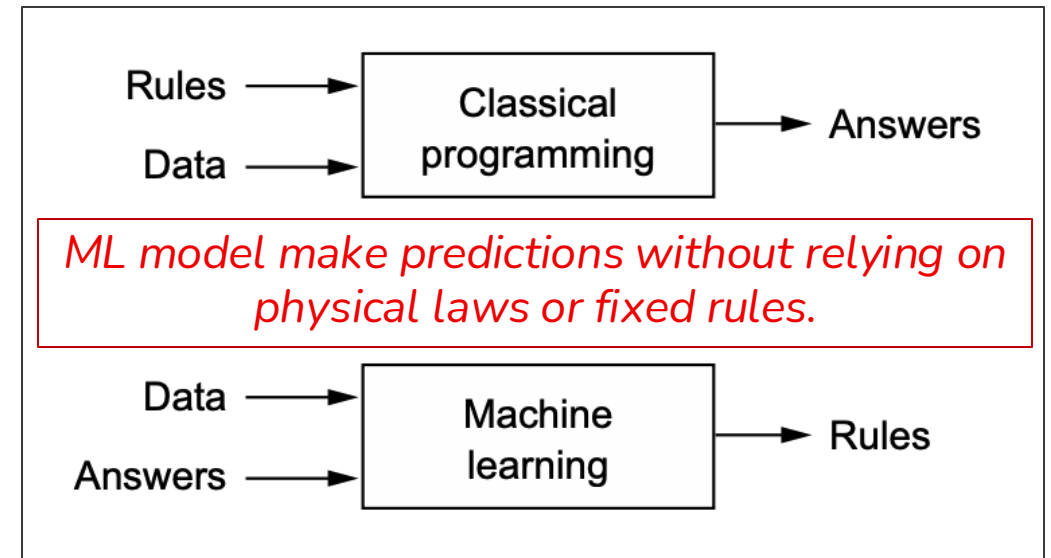
---

- *Research Background*
- *A Comprehensive Machine Learning Model-Building Workflow*
  - *Using Ozone Simulation as an Example*
- *A Machine Learning Starter Kit for Quick Application*

# Toward the AI Era: ML as a New Data-Driven Programming Paradigm



Machine learning: a new programming paradigm



*Data is no longer just a byproduct of processes, but a valuable asset in its own right.*

# The Inevitable Rise of ML in the Earth Science

Big data (e.g., satellites;  
ground-based sensors)

**Volume**  
Data size

**Velocity**  
Speed of  
change

**Variety**  
Diverse data  
sources

**Veracity**  
Uncertainty  
of data

“Knowledge from data”



Multiple pressing  
Environmental challenges



## 1. Data Explosion in Earth Science

- Environmental system data are exemplary of all four of the ‘four Vs’ of ‘big data’: *volume*, *velocity*, *variety* and *veracity*.

## 2. Complexity of Earth Systems

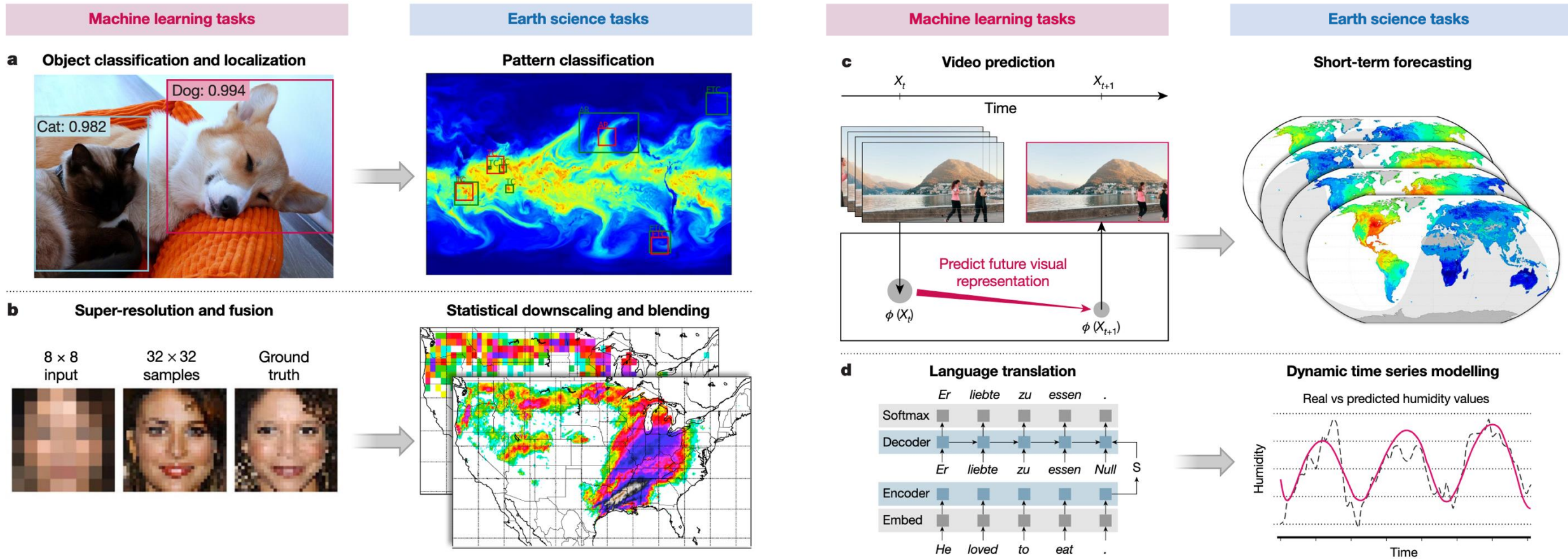
- Environmental systems are highly complex, nonlinear and interconnected. ML offers a complementary approach to model these intricate relationships where science is still incomplete.

## 3. Need for rapid and scalable solutions

- Lightweight; flexible input requirements; available plug-and-play models.

# Four Deep Learning Applications and Their Use Cases in Earth Science

Markus Reichstein et al., *Nature* (2019)



- Many central tasks in Earth Science are inherently well-suited to machine learning.



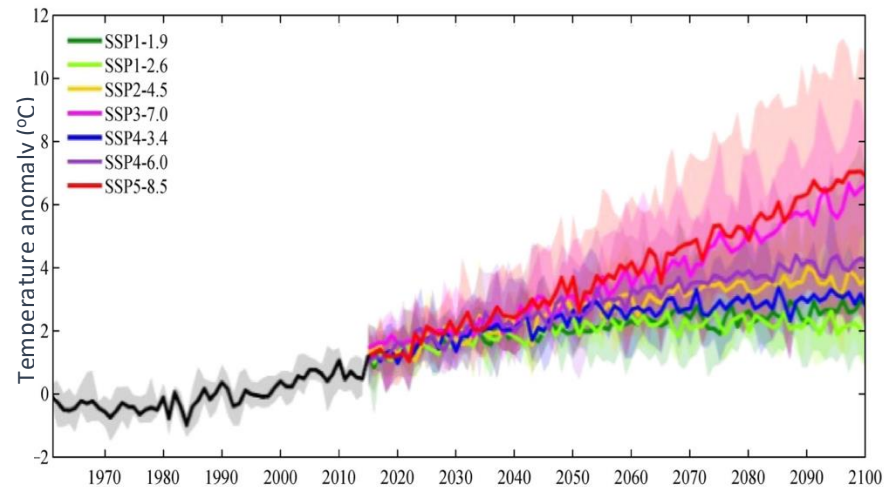
# Numerical Modeling Remains an Indispensable Tool for Understanding the Earth System

**Theory-Driven Numerical Modeling** is built on physical laws (e.g., conservation of mass, momentum, and energy), allowing scientists to simulate processes based on first principles.

This enables not just prediction but understanding of *why things happen*.

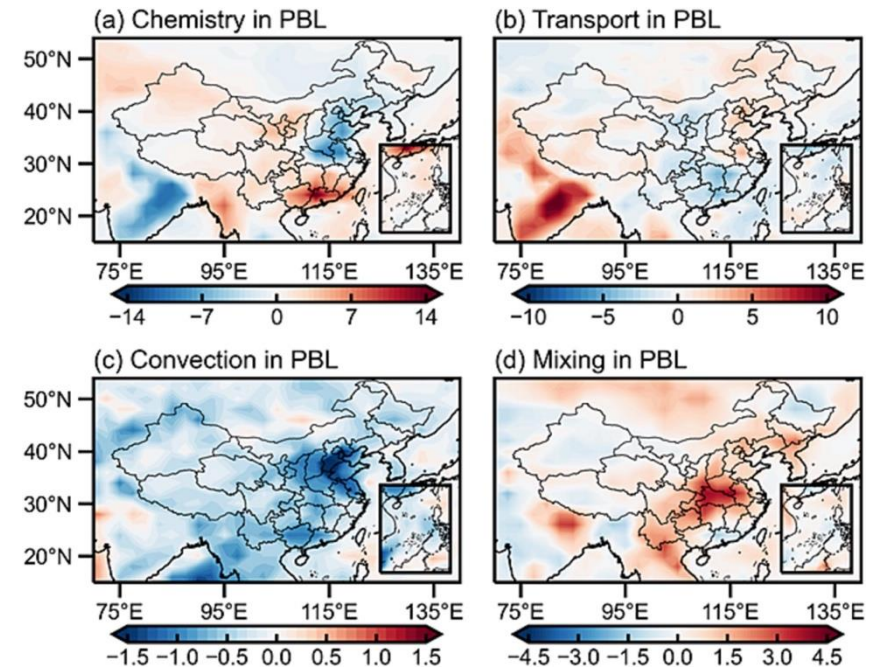
**Interpretability**

Changes in annual mean temperature during 1961–2100 under SSPs-RCPs (relative to 1995–2014)



Qin et al., *Front. Earth Sci* (2021)

Various processes contributing to warm season mean  $O_3$  mass change rates ( $kg\ s^{-1}$ )



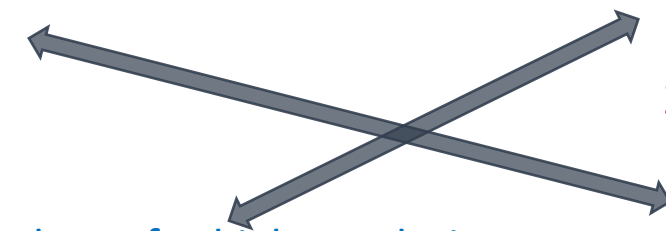
Ni et al., *STOTEN* (2024)

**1. Extrapolation beyond existing data (e.g. make predictions under specific scenarios)**

**2. Process attribution**

# A More Promising Direction: A Hybrid Modeling Approach

| Method        | Theory-driven Numerical Modelling                                                                                                                     | Data-driven ML                                                               |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>Nature</b> | Based on physical laws and equations                                                                                                                  | Based on data patterns and statistical inference                             |
| <b>Pros</b>   | 1. Interpretable                                                                                                                                      | 1. Lower computational cost<br>2. Flexible integration of heterogeneous data |
| <b>Cons</b>   | 1. High computational cost for high-resolution output<br>2. Large biases due to model and parameter uncertainties<br>3. Standard-compliant input data | 1. Lack of transparency and interpretability                                 |



**Historically**, they are treated as two different fields with very different scientific paradigms

The approaches complement each other: physical models are inherently interpretable and can extrapolate beyond observed conditions, while data-driven methods offer great flexibility in adapting to data.

# Diverse Applications of Hybrid Approaches in Environmental Science

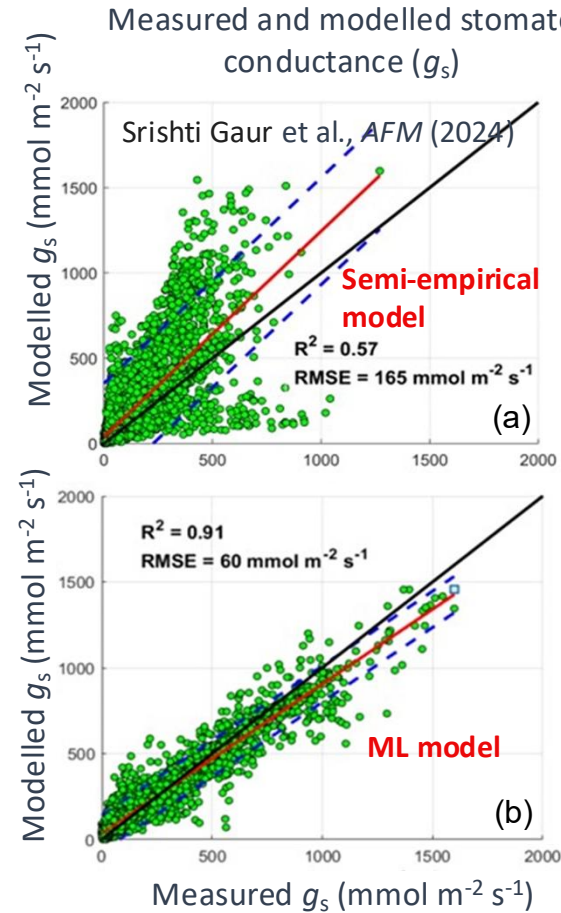
Climatic Change (2023) 176:101  
<https://doi.org/10.1007/s10584-023-03532-1>

Jebeile et al., *Climatic Change* (2023)

## Machine learning and the quest for objectivity in climate model parameterization

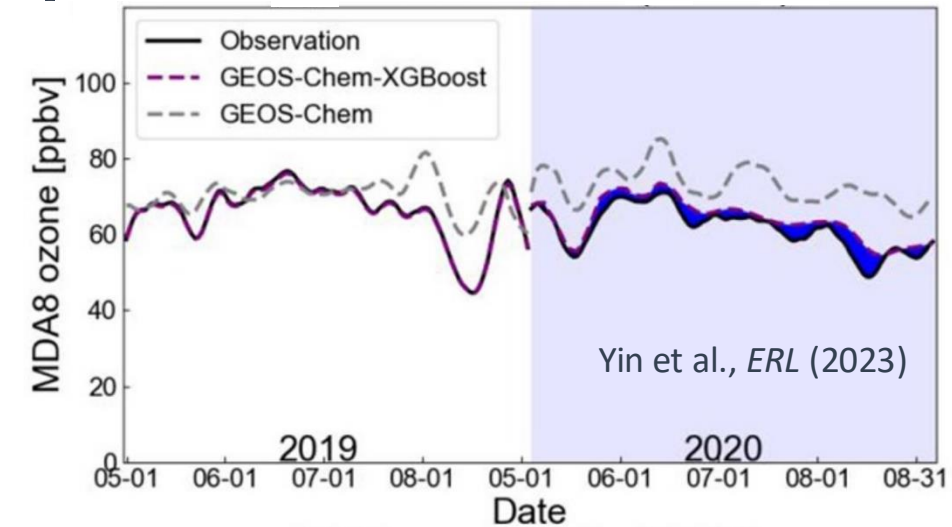
Julie Jebeile<sup>1,2,3</sup> · Vincent Lam<sup>1,2,4</sup> · Mason Majszak<sup>1,2</sup>  · Tim Rätz<sup>1</sup>

- Hydrologic model parameters
- Convection parameterization
- ...



In practice, it can serve to correct the model bias of dynamic variables, or facilitate improved downscaling to finer spatial scales

Observational and simulated MDA8 O<sub>3</sub>



1. Improving parameterizations

2. Replacing a 'physical' sub-model with a ML model

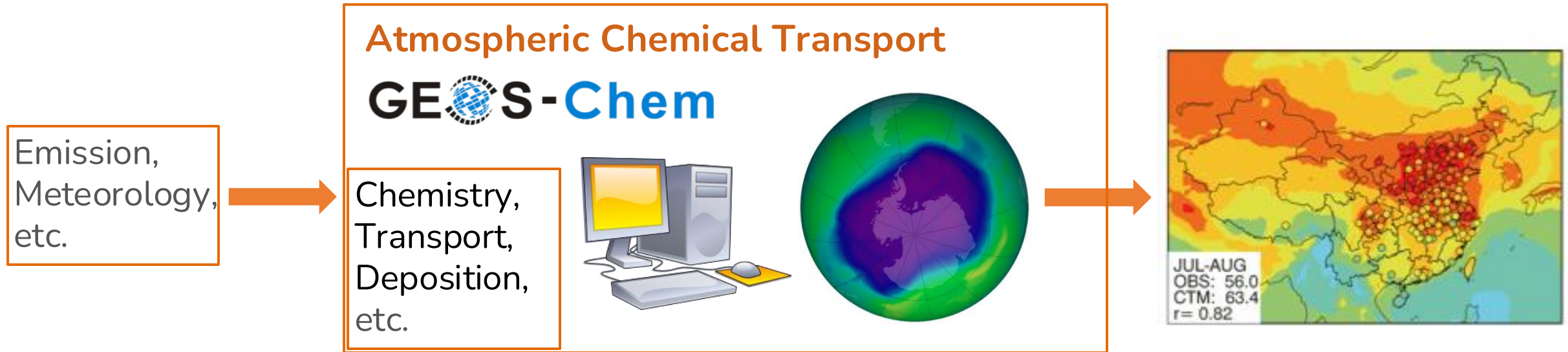
3. Analysis of model–observation mismatch

*Knowing how to use a tool is a skill; knowing when and where to apply it to solve critical problems is an art—an art in which imagination draws from a deep well of knowledge.*



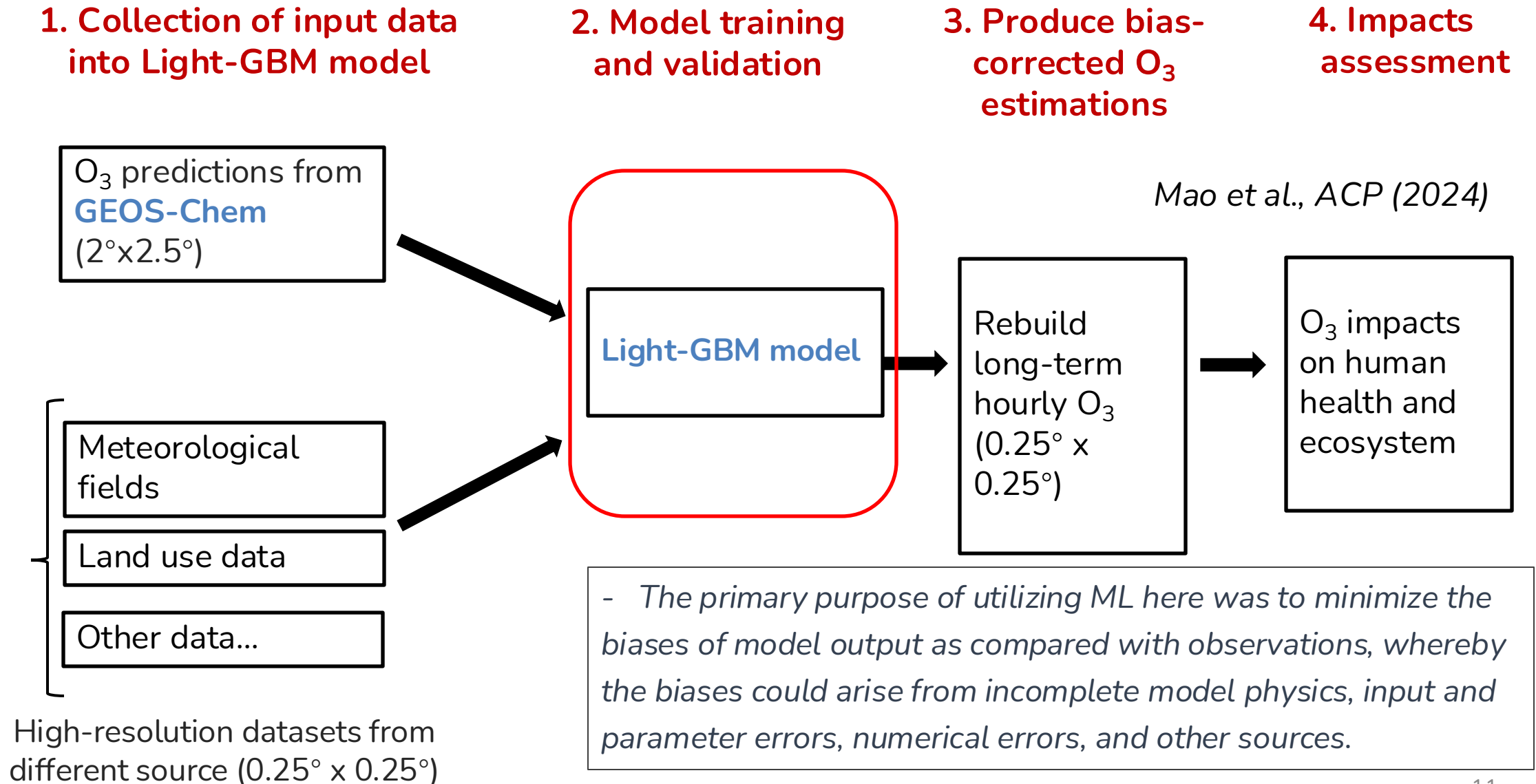
- *Research Background*
- *A Comprehensive Machine Learning Model-Building Workflow*
  - *Using Ozone Simulation as an Example*
- *A Machine Learning Starter Kit for Quick Application*

# Chemical Transport Model as the Traditional Tool to Simulate Air Pollutant



- High computational cost for high-resolution simulation
- Large biases exist due to uncertainties (e.g. emission inventories, parameterization of physical and chemical processes)

# Workflow: A Hybrid Approach Combining GEOS-Chem and ML

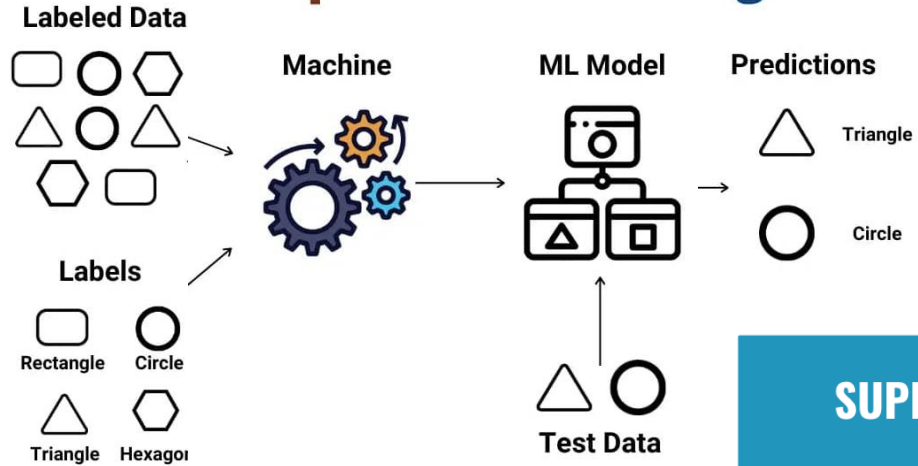


## *Key Steps in the Building a Machine Learning Model*

1. Problem Type Identification
2. Data Preparation
3. Model Training
4. Model Assessment
5. Model Optimization

# 1. Problem Type Identification— Supervised, Unsupervised or Reinforcement?

## Supervised Learning



learn from labels



- Decision tree
- Support vector machines
- ...



- Linear regression,
- Random forest,
- Support vector regression
- ...



learn without labels



- K-means clustering
- Mean shift clustering
- ...



learn from experience



- Q-learning
- R-learning
- ...

The purpose of Problem Type Identification is to help you understand which ML models are applicable.



## 2. Data Preparation — Standard Input Format for ML models

```
train_D.head(10)
```

| Samples             | geos_o3 | NOx   | VOC   | CO     | mon | hour | landuse | topo  | gdp | pop   | t2m   | u10m | v10m | ssrd      | msl    | rh   | obs_o3 |
|---------------------|---------|-------|-------|--------|-----|------|---------|-------|-----|-------|-------|------|------|-----------|--------|------|--------|
| 2016-01-01 08:00:00 | 2.6     | 441.2 | 696.9 | 5284.3 | 1   | 8    | 2       | 275.0 | 5   | 145.4 | 264.7 | 1.1  | -1.1 | 11112.2   | 1024.7 | 68.5 | 2.0    |
| 2016-01-01 09:00:00 | 6.2     | 441.2 | 696.9 | 5284.3 | 1   | 9    | 2       | 275.0 | 5   | 145.4 | 265.2 | 1.2  | -1.3 | 314092.7  | 1025.0 | 65.2 | 2.0    |
| 2016-01-01 10:00:00 | 9.0     | 441.2 | 696.9 | 5284.3 | 1   | 10   | 2       | 275.0 | 5   | 145.4 | 272.3 | 0.9  | -1.0 | 819271.4  | 1024.6 | 46.8 | 3.0    |
| 2016-01-01 11:00:00 | 11.9    | 441.2 | 696.9 | 5284.3 | 1   | 11   | 2       | 275.0 | 5   | 145.4 | 273.9 | 0.5  | -0.3 | 1250058.5 | 1024.0 | 43.3 | 7.0    |
| 2016-01-01 12:00:00 | 14.9    | 441.2 | 696.9 | 5284.3 | 1   | 12   | 2       | 275.0 | 5   | 145.4 | 275.3 | 0.5  | 0.5  | 1516843.2 | 1022.2 | 43.8 | 13.0   |
| 2016-01-01 13:00:00 | 16.9    | 441.2 | 696.9 | 5284.3 | 1   | 13   | 2       | 275.0 | 5   | 145.4 | 278.0 | 0.5  | 0.9  | 1583193.4 | 1021.0 | 32.8 | 9.0    |
| 2016-01-01 14:00:00 | 16.7    | 441.2 | 696.9 | 5284.3 | 1   | 14   | 2       | 275.0 | 5   | 145.4 | 278.2 | 0.7  | 1.2  | 1440191.1 | 1019.6 | 33.3 | 16.0   |
| 2016-01-01 15:00:00 | 13.9    | 441.2 | 696.9 | 5284.3 | 1   | 15   | 2       | 275.0 | 5   | 145.4 | 278.0 | 0.0  | 1.2  | 1104292.2 | 1019.9 | 36.0 | 17.0   |
| 2016-01-01 16:00:00 | 9.5     | 441.2 | 696.9 | 5284.3 | 1   | 16   | 2       | 275.0 | 5   | 145.4 | 275.3 | -0.4 | 1.3  | 627666.9  | 1019.9 | 50.5 | 12.0   |
| 2016-01-01 17:00:00 | 6.0     | 441.2 | 696.9 | 5284.3 | 1   | 17   | 2       | 275.0 | 5   | 145.4 | 276.1 | -0.2 | 0.8  | 146954.7  | 1020.5 | 47.6 | 2.0    |

(years x days x hours x sites)

Predictors (features)

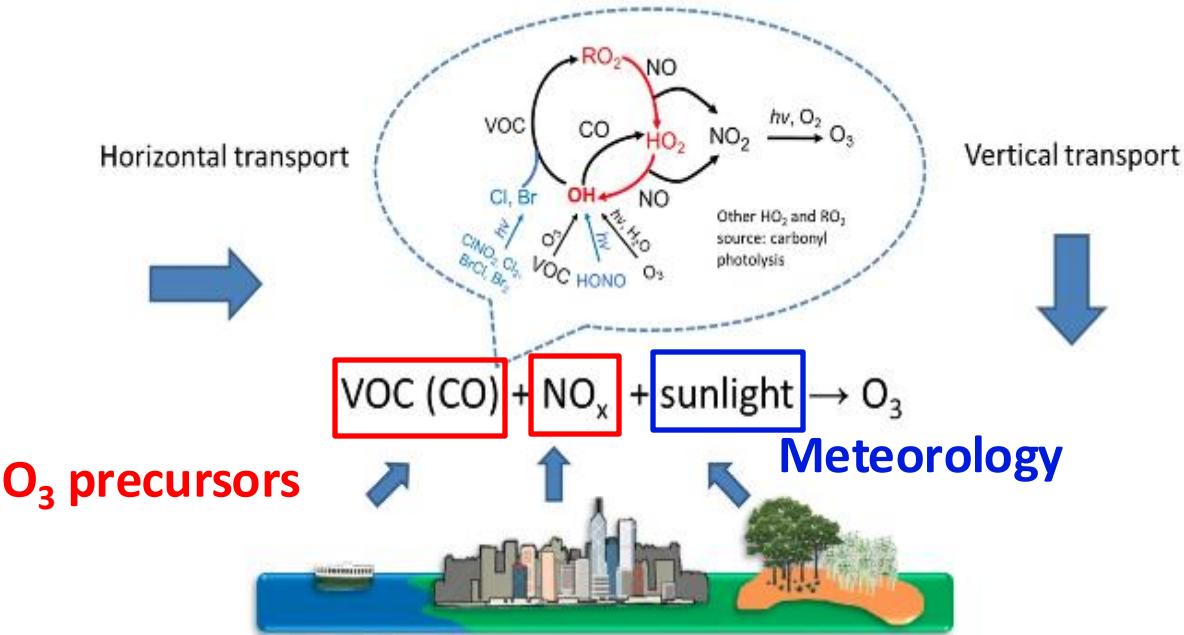
Predictand (labels)

The input data for a machine learning model is typically a two-dimensional table, consisting of **predictors (features)** and **predictands (targets or labels)** for supervised problem.

# 2. Data Preparation — Knowledge-based Input Feature Selection

## Secondary air pollutant

(not directly emitted from emission sources)



Wang et al., ERL (2022)

| Input data                                                                             |                                                                                   |
|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Meteorology</b><br><br>SSRD, SLP,<br>T_2m, RH, TP,<br>U_10m, U_850,<br>V_10m, V_850 | <b>GSEO-Chem O<sub>3</sub></b>                                                    |
| <b>Land data</b><br><br>Land use type,<br>Topography                                   | <b>Anthropogenic emissions (MEIC)</b><br><br>CO, NO <sub>x</sub> , and<br>NMVOCs, |
|                                                                                        | <b>Auxiliary data</b><br><br>GDP, Pop, MOY,<br>HOD                                |

High anthropogenic emissions and unfavorable weather

## 2. Data Preparation — Clean Missing Values

1) A Simple Option: Drop Columns/Rows

| Site1 | Site2 |
|-------|-------|
| 61.0  | 71.0  |
| 59.0  | 73.0  |
| 43.0  | 65.0  |
| NaN   | 66.0  |



| Site2 |
|-------|
| 71.0  |
| 73.0  |
| 65.0  |
| 66.0  |

2) A Better Option: Imputation (e.g.. mean value)

| Site1 | Site2 |
|-------|-------|
| 61.0  | 71.0  |
| 59.0  | 73.0  |
| 43.0  | 65.0  |
| NaN   | 66.0  |



| Site1 | Site2 |
|-------|-------|
| 61.0  | 71.0  |
| 59.0  | 73.0  |
| 43.0  | 65.0  |
| 54.3  | 66.0  |

Most machine learning libraries give an error if you try to build a model using data with missing values.

## 2. Data Preparation — Clean Categorical Variables

### 1) Ordinal Encoding

| Land Types |   |
|------------|---|
| Forest     | 1 |
| Shrub      | 2 |
| Grassland  | 3 |
| Urban      | 4 |
| Cropland   | 5 |

- Assigning each unique value to a different integer

### 2) One-Hot Encoding

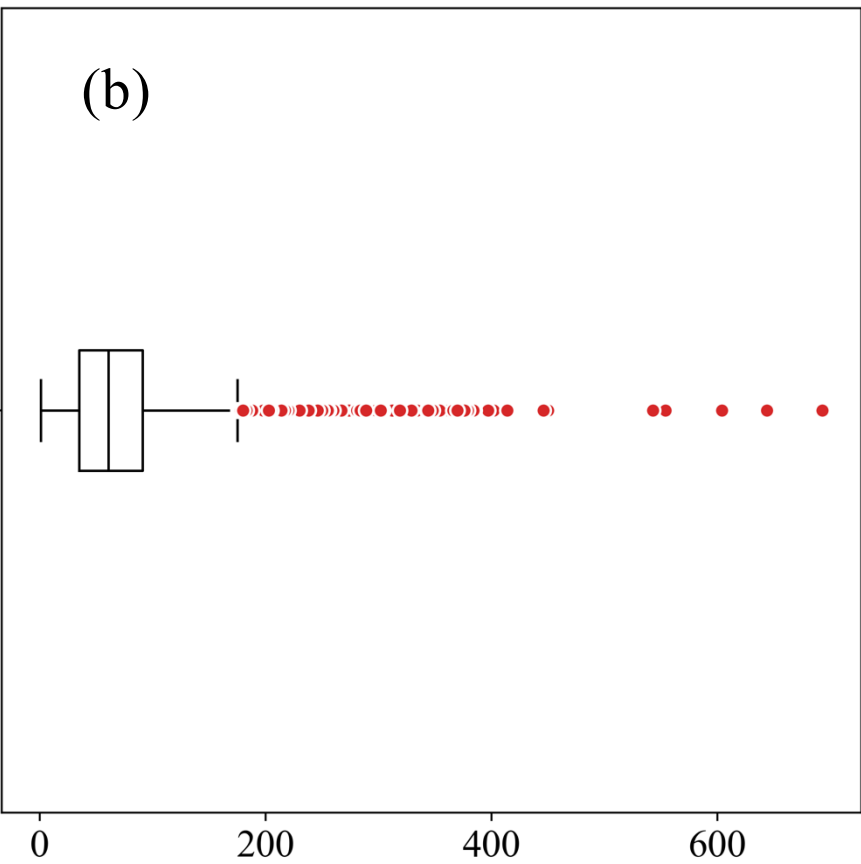
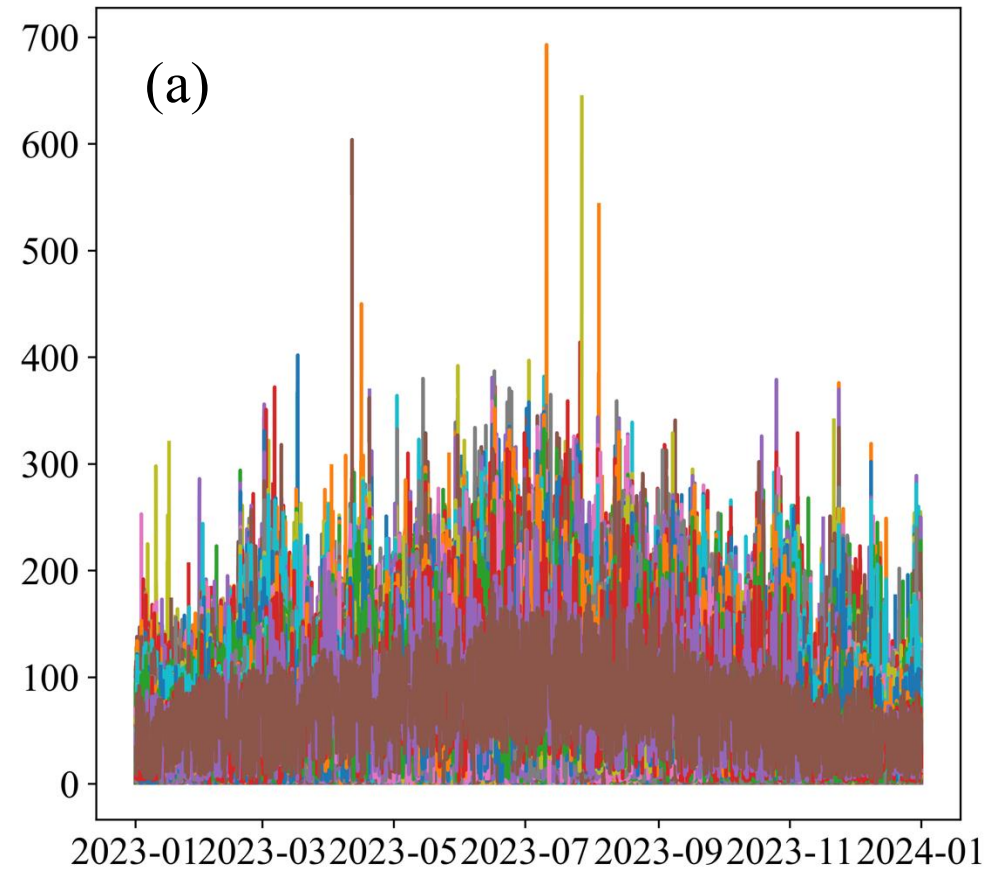
| Land Types |  |
|------------|--|
| Forest     |  |
| Shrub      |  |
| Grassland  |  |
| Urban      |  |
| Cropland   |  |

| Forest | Shrub | Urban |
|--------|-------|-------|
| 1      | 0     | 0     |
| 0      | 1     | 0     |
| 0      | 0     | 0     |
| 0      | 0     | 1     |
| 0      | 0     | 0     |

- Creating new columns indicating the presence of each possible value in the original data (work particularly well if there is no clear ordering in the categorical data).

You will get an error if you try to plug these variables into **most ML models** without preprocessing them first.

## 2. Data Preparation — Clean Abnormal Values



### 3. Model Training – Model Creation and Fitting

```
train_D.head(10)
```

|                     | geos_o3 | NOx   | VOC   | CO     | mon | hour | obs_o3 |
|---------------------|---------|-------|-------|--------|-----|------|--------|
| 2016-01-01 08:00:00 | 2.6     | 441.2 | 696.9 | 5284.3 | 1   | 8    | 2.0    |
| 2016-01-01 09:00:00 | 6.2     | 441.2 | 696.9 | 5284.3 | 1   | 9    | 2.0    |
| 2016-01-01 10:00:00 | 9.0     | 441.2 | 696.9 | 5284.3 | 1   | 10   | 3.0    |
| 2016-01-01 11:00:00 | 11.9    | 441.2 | 696.9 | 5284.3 | 1   | 11   | 7.0    |
| 2016-01-01 12:00:00 | 14.9    | 441.2 | 696.9 | 5284.3 | 1   | 12   | 13.0   |
| 2016-01-01 13:00:00 | 16.9    | 441.2 | 696.9 | 5284.3 | 1   | 13   | 9.0    |
| 2016-01-01 14:00:00 | 16.7    | 441.2 | 696.9 | 5284.3 | 1   | 14   | 16.0   |
| 2016-01-01 15:00:00 | 13.9    | 441.2 | 696.9 | 5284.3 | 1   | 15   | 17.0   |
| 2016-01-01 16:00:00 | 9.5     | 441.2 | 696.9 | 5284.3 | 1   | 16   | 12.0   |
| 2016-01-01 17:00:00 | 6.0     | 441.2 | 696.9 | 5284.3 | 1   | 17   | 2.0    |

Features (X)

Labels (Y)

```
import lightgbm
```

```
# 1) Create a model
```

```
model = lightgbm.LGBMRegressor(random_state=1)
```

```
# 2) Train the model
```

```
fitted_model = model.fit(X, y)
```

```
# 3) Make predictions
```

```
predicted_y = fitted_model.predict(X)
```

#### ● Note:

Many ML models include elements of randomness (e.g., bootstrapping in Random Forest). Setting **random\_state** ensures reproducibility—i.e., the same results every time you run the code.



## 4. Model Assessment – Evaluation Metrics

### 1. Mean Absolute Error (MAE)

$$\text{MAE} = \frac{\sum_{i=1}^N |y_i - \hat{y}_i|}{N}$$

### 2. Mean Squared Error(MSE)

$$\text{MSE} = \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{N}$$

### 3. Root Mean Squared Error(RMSE)

$$\text{RMSE} = \sqrt{\frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{N}}$$

### 4. R-squared ( $R^2$ ) Score

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$$

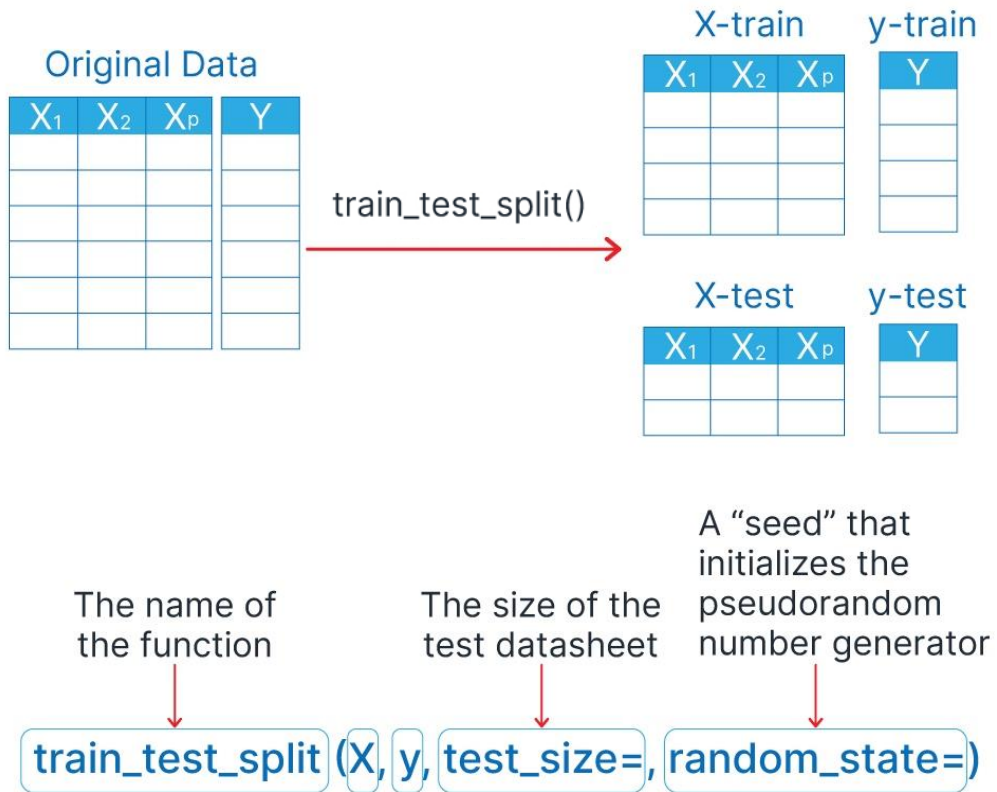
$y_i$  and  $\hat{y}_i$  represent the actual and predicted values for the  $i$ -th sample, respectively.  $\bar{y}$  is the mean of all true values, and  $N$  is the total number of samples.

## 4. Model Assessment – Training and Test Data



### Note:

Training data is used to build the model, while test data is used to evaluate the model's **generalization ability** on previously unseen data.



```
import lightgbm
```

```
# 1) Split training and test data
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3)
```

```
# 2) Create the model
```

```
model = lightgbm.LGBMRegressor(random_state=1)
```

```
# 3) Train the model on training data
```

```
fitted_model = model.fit(X_train, y_train)
```

```
# 4) Make predictions on test data
```

```
predicted_y_test = fitted_model.predict(X_test)
```

```
# 5) Model evaluation
```

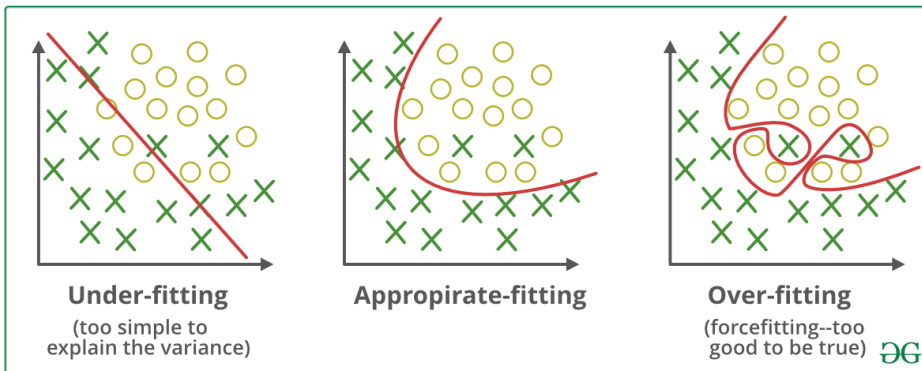
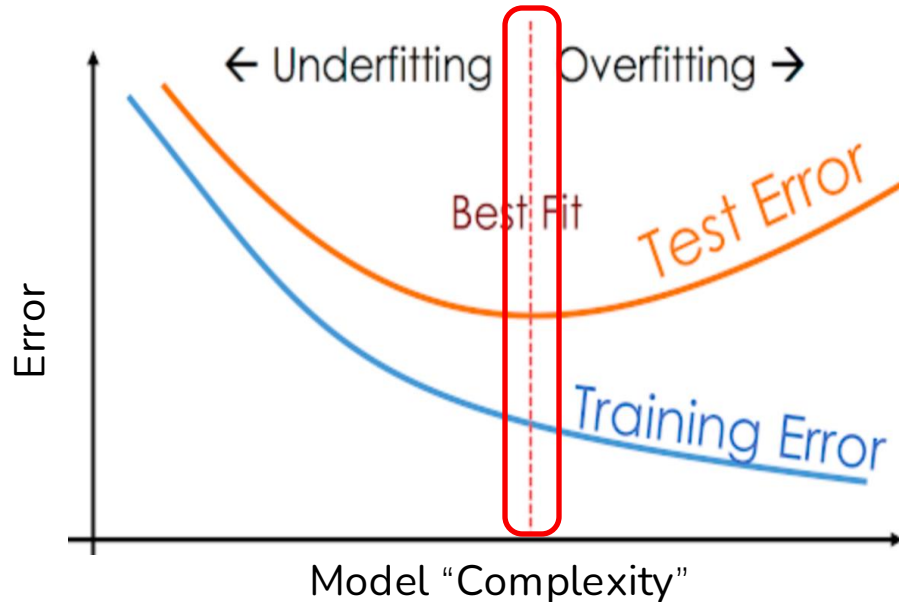
```
rmse = cal_rmse(predicted_y_test, obs_y)
```

```
r2 = cal_r2(predicted_y_test, obs_y)
```

```
print(rmse, r2)
```

## 4. Model Assessment — Overfitting and Underfitting

A Good ML Model = Accuracy + Strong Generalization



### Ways to Reduce Underfitting

- Increase model complexity
- Add more features (feature engineering)
- Train longer
- ...

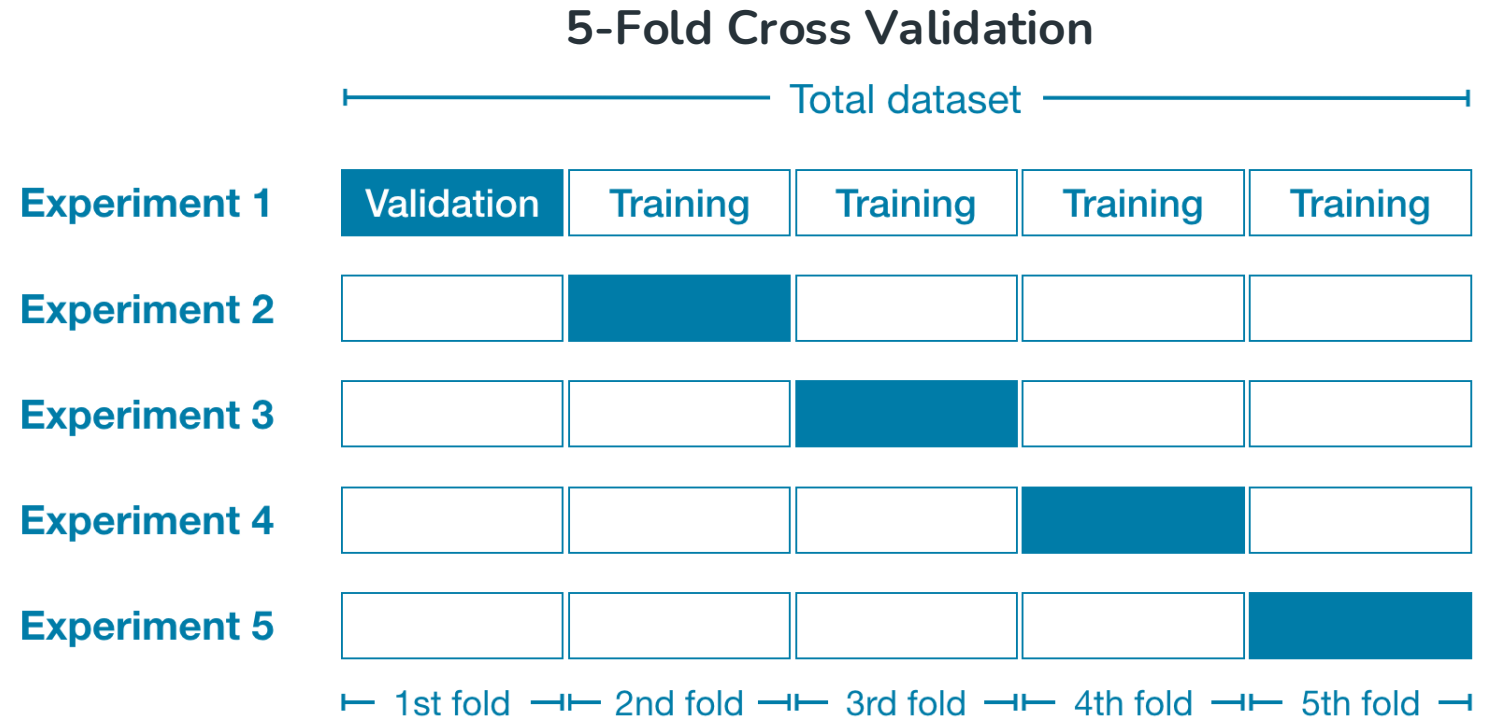
### Ways to Reduce Overfitting

- Reduce model complexity
  - Improving the quality of training data, mitigating the risk of fitting the noise
  - Increase the training data
  - ...
- *An expected good model achieves high accuracy through strong generalization—not memorization—demonstrating its ability to capture the true patterns in the data.*

## 5. Model Optimization — Cross Validation (Generalization)

- Cross-validation is a technique used to check how well a ML model performs on unseen data (generalizability).

- Cross-validation evaluates model performance by repeatedly splitting the training data. In **k-fold** cross-validation, the data is split into k parts; each part serves once as validation while the rest train the model. To achieve consistently high scores across folds, the model must generalize well to all samples.



### Pros:

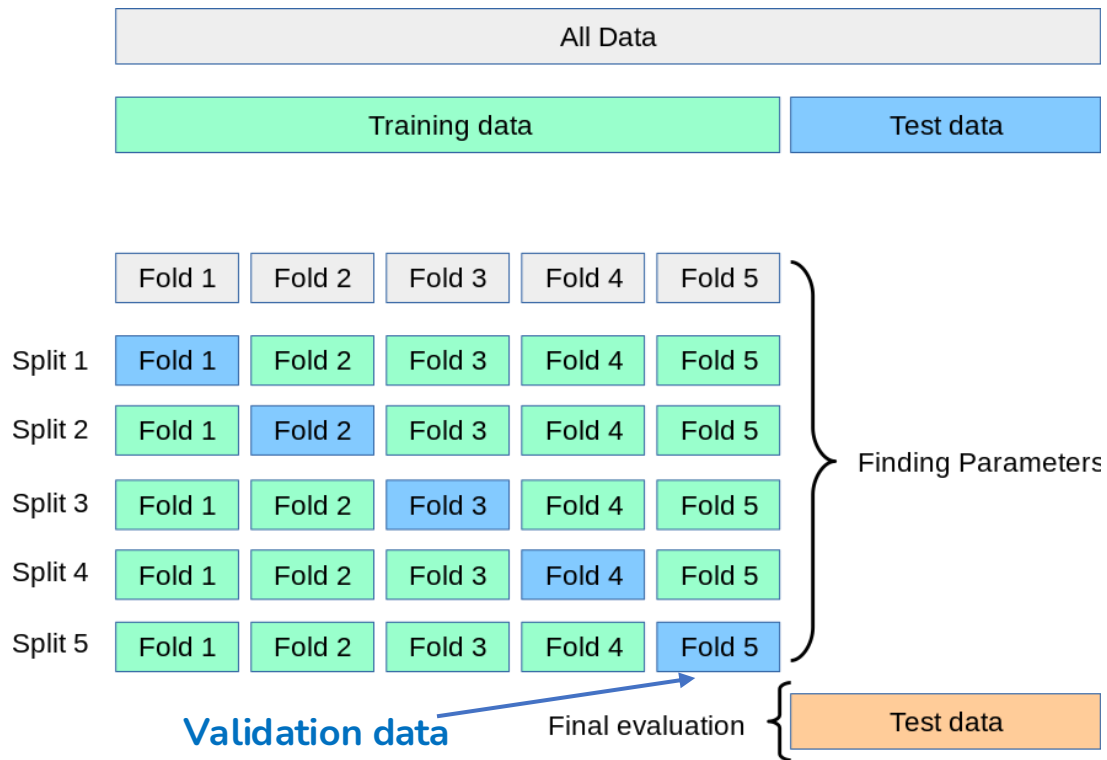
- 1) Higher data utilization
- 2) More robust and reliable performance estimates

### Cons:

- 1) Increased computational cost due to multiple training runs

# 5. Model Optimization – Cross Validation (Generalization)

## 5-Fold Cross Validation



### 4.2.1 训练集、验证集和测试集

评估模型的重点是将数据划分为三个集合：训练集、验证集和测试集。在训练数据上训练模型，在验证数据上评估模型。一旦找到了最佳参数，就在测试数据上最后测试一次。

你可能会问，为什么不是两个集合：一个训练集和一个测试集？在训练集上训练模型，然后在测试集上评估模型。这样简单得多！

原因在于开发模型时总是需要调节模型配置，比如选择层数或每层大小 [这叫作模型的超参数 (hyperparameter)，以便与模型参数 (即权重) 区分开]。这个调节过程需要使用模型在验证数据上的性能作为反馈信号。这个调节过程本质上就是一种学习：在某个参数空间中寻找良好的模型配置。因此，如果基于模型在验证集上的性能来调节模型配置，会很快导致模型在验证集上过拟合，即使你并没有在验证集上直接训练模型也会如此。

造成这一现象的关键在于信息泄露 (information leak)。每次基于模型在验证集上的性能来调节模型超参数，都会有一些关于验证数据的信息泄露到模型中。如果对每个参数只调节一次，那么泄露的信息很少，验证集仍然可以可靠地评估模型。但如果你多次重复这一过程 (运行一次实验，在验证集上评估，然后据此修改模型)，那么将会有越来越多的关于验证集的信息泄露到模型中。

最后，你得到的模型在验证集上的性能非常好 (人为造成的)，因为这正是你优化的目的。你关心的是模型在全新数据上的性能，而不是在验证数据上的性能，因此你需要使用一个完全不同的、前所未见的数据集来评估模型，它就是测试集。你的模型一定不能读取与测试集有关的任何信息，既使间接读取也不行。如果基于测试集性能来调节模型，那么对泛化能力的衡量是不准确的。

- Model development involves hyperparameter tuning (such as tree depth or leaf nodes), and this process essentially “learns” from the data through the feedback from validation data (indirect way), leading to possible data leakage. Therefore, we keep a separate unseen test data for final assessment of performance.

## 5. Model Optimization – Data Quality and Model Tuning (Accuracy)

### 1. Data optimization

— Improving the quality, relevance, and representation of the input data.

#### 1) Feature engineering

- Add meaningful features

#### 2) Feature selection

- Remove irrelevant or redundant features

#### 3) Normalization/Standardization

- Scale features appropriately for algorithms sensitive to feature scales

...

### 2. Model optimization

— Tuning hyperparameters, selecting architectures, applying regularization, etc.

#### 1) Model Selection

#### 2) Hyperparameter Tuning

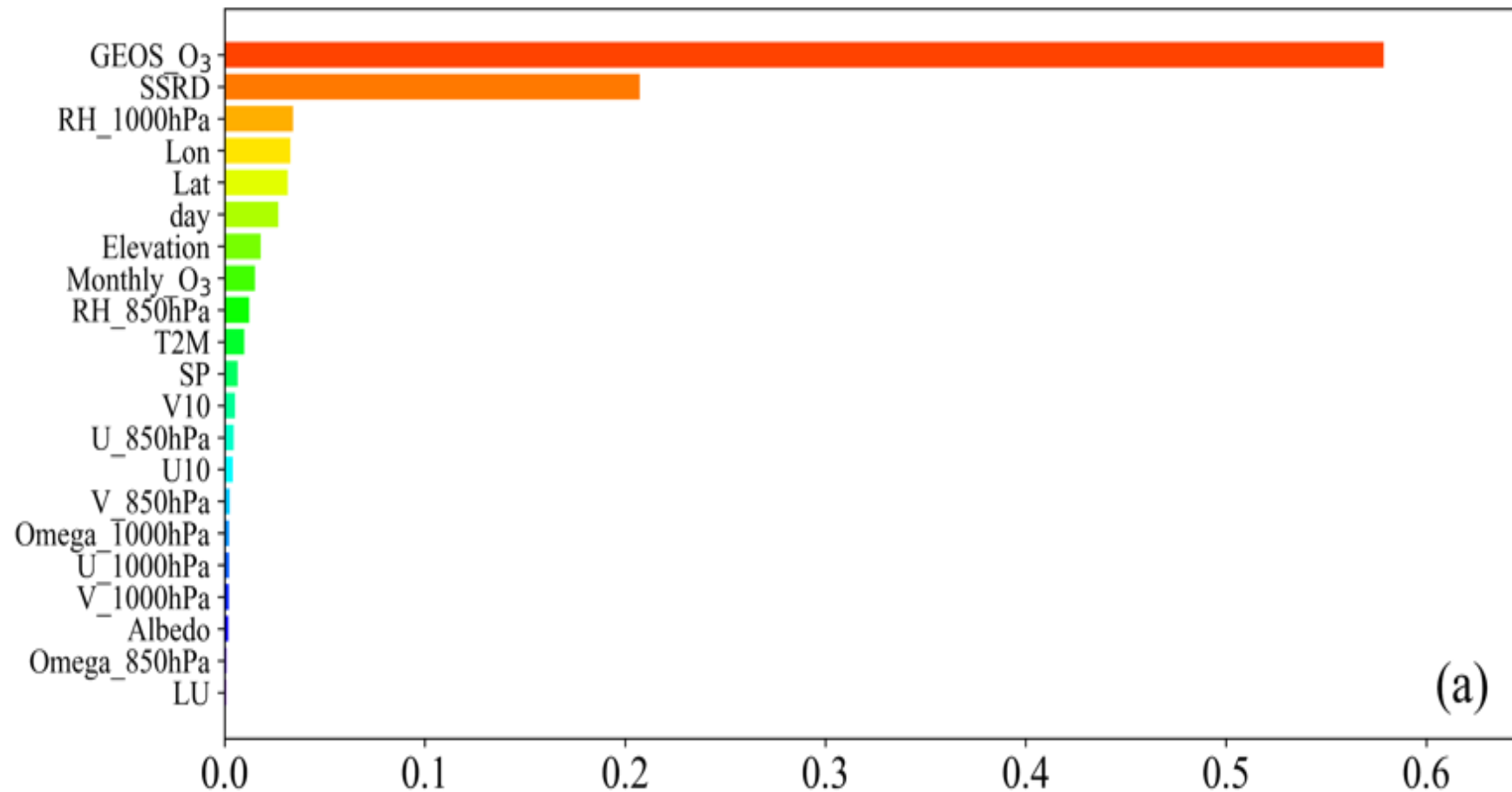
#### 3) Regularization

#### 4) Ensemble Methods

...

- “Garbage in, garbage out”: high-quality, well-prepared data is the foundation of any good model. Optimizing data often yields the biggest improvements with relatively simple effort.

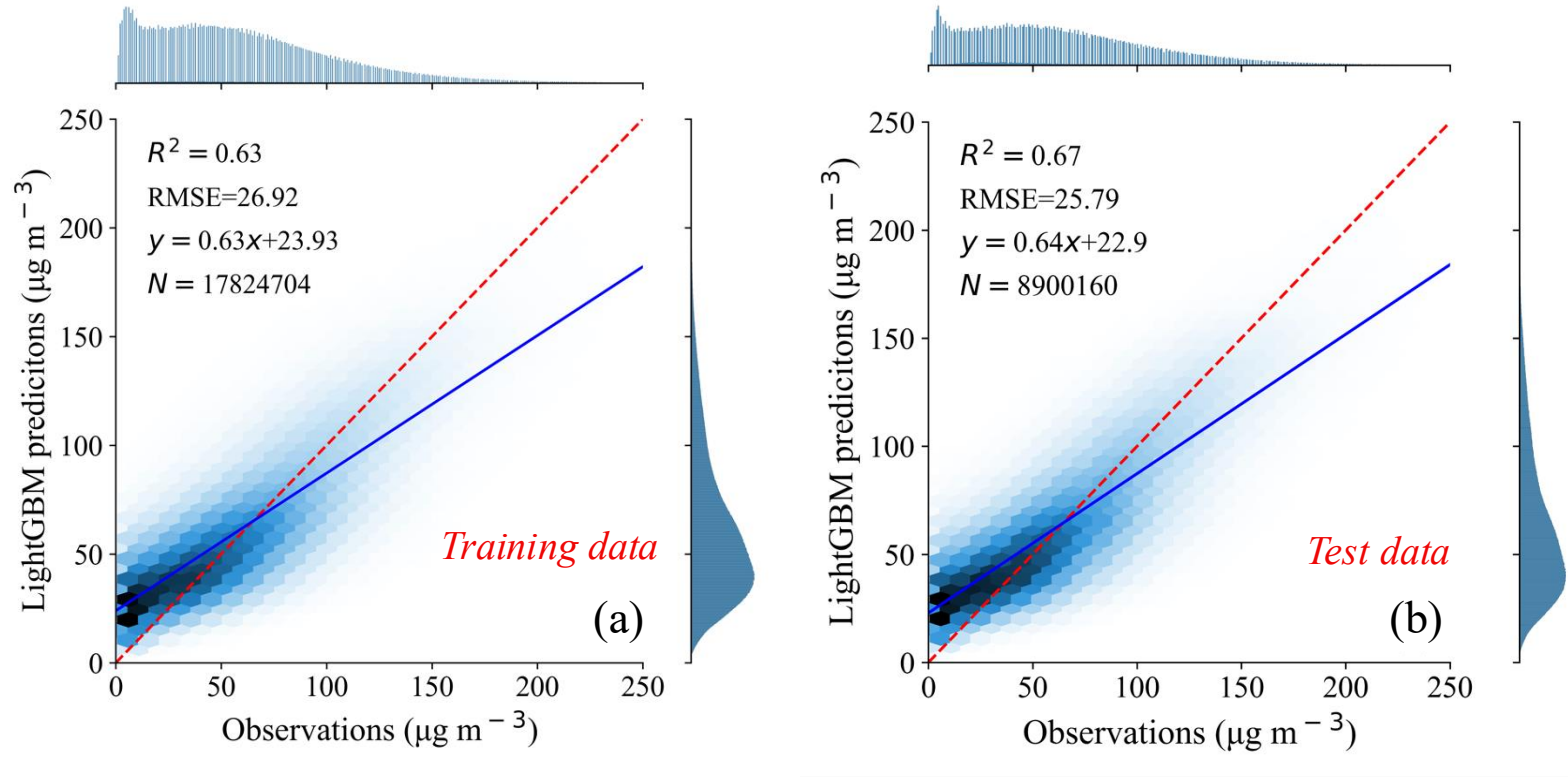
## 5. Model Optimization – Feature Importance Plot



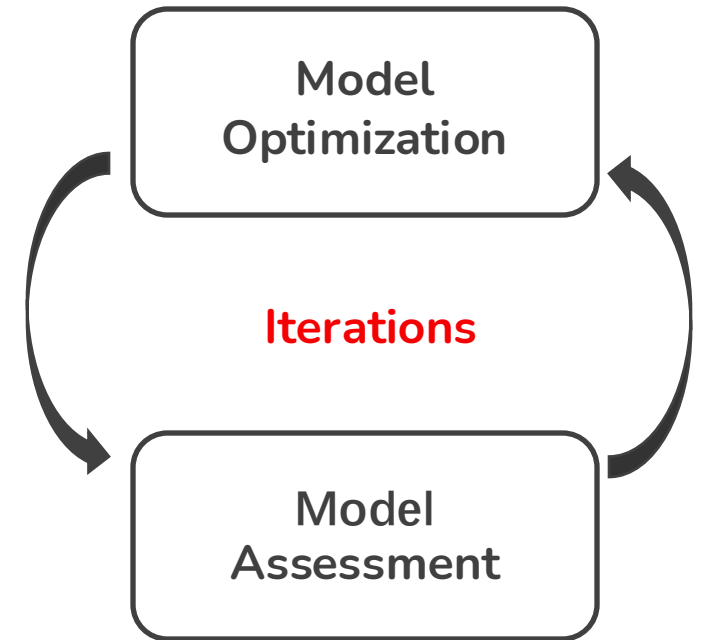
- Feature importance reveals which features the model relies on but does not uncover true causal relationships in the data. That is to say, feature importance provides interpretability from the model perspective, not mechanistic understanding of the system.



# When to Finalize the Model



A well-performing model with strong generalization capability



- ***Research Background***
- ***A Comprehensive Machine Learning Model-Building Workflow***
  - *Using Ozone Simulation as an Example*
- ***A Machine Learning Starter Kit for Quick Application***

# A ML Starter Kit for Quick Application in Hourly Ozone Prediction

## Requirements

- Python 3.8 or above
- Key Python packages: numpy, pandas, scikit-learn, xgboost, matplotlib, shap, etc. (see environment.yml for details)
- CDO (Climate Data Operators) command-line tool (see 3\_CDO\_tutorial/ for installation and usage)

## Getting Started

### 1. Clone the repository:

```
git clone https://github.com/maojia527/ML-AtmosStarter.git  
cd ML-AtmosStarter
```

### 2. Set up the Conda environment:

```
conda env create -f environment.yml  
conda activate atmos-ml
```

Installs all dependencies and ensures the environment can be consistently reproduced across other machines or users, relative to mine.

### 3. Run the quick start notebook with well-prepared input data:

In Jupyter Notebook or JupyterLab to walk through the full ML workflow: *model\_building\_workflow.ipynb*



## Further Reading

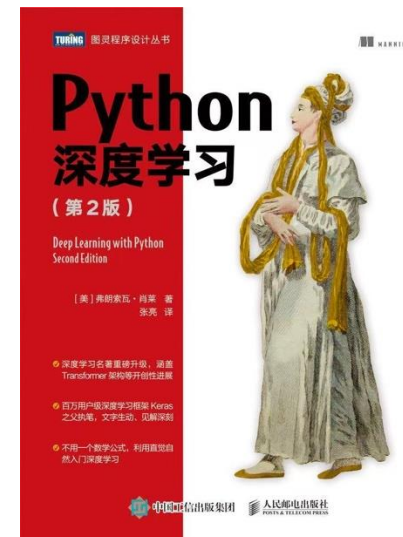
- **Book: Deep Learning with Python**

- **ML-related Issues**

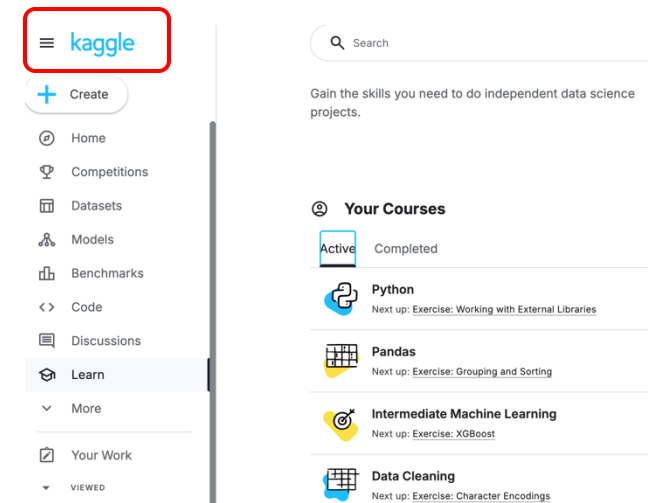
- 1) Data Leakage
- 2) Underfitting and Overfitting

- **ML-related Techniques**

- 1) Cross Validation
- 2) Pipeline
- 3) GridSearch
- 4) Hyperparameter tuning



## Beginner-Friendly Online Courses



<https://www.kaggle.com/learn>

<https://lightgbm.readthedocs.io/en/latest/Parameters-Tuning.html>



# Sources of Raw Data

- Air pollutant data from monitoring stations in China:

<https://quotsoft.net/air/>

- Meteorological gridded data (ERA5 reanalysis; registration required):

<https://cds.climate.copernicus.eu/>

- Gridded precursor emissions data (MEIC; account application required): CEDS

[http://meicmodel.org.cn/?page\\_id=125&lang=en](http://meicmodel.org.cn/?page_id=125&lang=en)

- Gridded population data:

<https://zenodo.org/records/10088105>

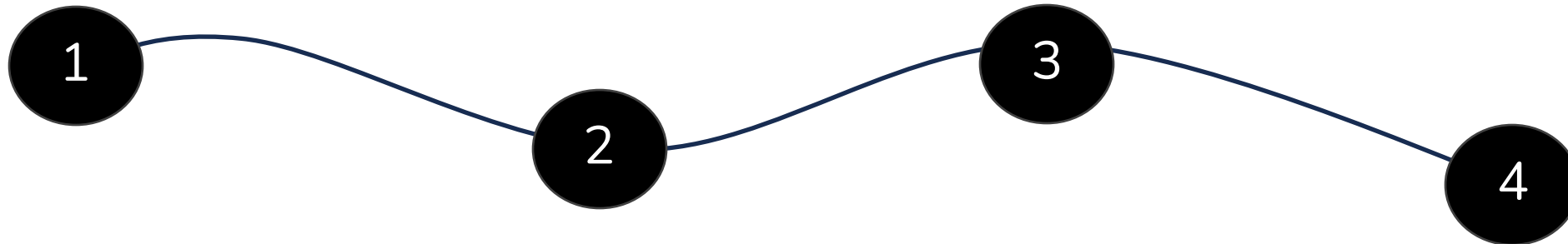
- Gridded GDP data:

[https://figshare.com/articles/dataset/Global\\_1\\_km\\_1\\_km\\_gridded\\_revised\\_real\\_gross\\_domestic\\_product\\_and\\_electricity\\_consumption\\_during\\_1992-2019\\_based\\_on\\_calibrated\\_nighttime\\_light\\_data/17004523/1](https://figshare.com/articles/dataset/Global_1_km_1_km_gridded_revised_real_gross_domestic_product_and_electricity_consumption_during_1992-2019_based_on_calibrated_nighttime_light_data/17004523/1)

# 🌟 A Recommended Quick Learning Path to Master ML Modeling

**1. Review the ppt** to build a fundamental understanding of machine learning concepts.

**3. Refer to further readings** for parts of `model_building_workflow.ipynb` that are unclear or need deeper understanding.



**2. Directly work on `model_building_workflow.ipynb`** to gain hands-on experience and better understand the input data and modeling details.

**4. Try building your own model from scratch** (e.g., using the provided raw data to build daily  $\text{PM}_{2.5}$  predictions).

# **A Brief Introduction to CDO: Efficient Processing of Reanalysis Data**



## 1. What is CDO?

- CDO is a collection of command line Operators to manipulate and analyse Climate and NWP model Data. Supported data formats includes **GRIB 1/2, netCDF 3/4, and etc.**

## 2. What can CDO do?

| Category                   | Function Description                                         | Example Commands                          |
|----------------------------|--------------------------------------------------------------|-------------------------------------------|
| Information                | Inspect metadata and data structure                          | cdo info, cdo sinfo, cdo ninfo            |
| Selection                  | Extract subsets based on variables, levels, regions, or time | cdo selvar, cdo seldate, cdo sellonlatbox |
| File Operations            | Merge, split, append, or convert files                       | cdo cat, cdo mergetime, cdo splitname     |
| Arithmetic & Statistics    | Apply mathematical or statistical operations on data         | cdo add, cdo sub, cdo timmean, cdo fldmax |
| Regridding & Interpolation | Regrid data to new spatial resolutions                       | cdo remapbil, cdo remapnn                 |
| Climate Diagnostics        | Compute climate indices or perform climate diagnostics       | cdo etccdi, cdo seasmean                  |

| Command Example                                                   | Explanation                                                                                                                                                                                                                                                                                                             |
|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>cdo info input.nc; cdo show date; cdo show griddes2</code>  | Show file info: variables, dimensions, time range, and grid info.                                                                                                                                                                                                                                                       |
| <code>cdo merge infile1.nc infile2.nc outfile.nc</code>           | Combines files for different variables with matching grids                                                                                                                                                                                                                                                              |
| <code>cdo copy infile1.nc infile2.nc outfile.nc</code>            | Combines files with matching grids along the time dimension.                                                                                                                                                                                                                                                            |
| <code>cdo seldate,2001-01-01,2001-12-31 input.nc output.nc</code> | Extracts data between specified dates.                                                                                                                                                                                                                                                                                  |
| <code>cdo daymean input.nc output_daymean.nc</code>               | Calculates daily average from sub-daily data.                                                                                                                                                                                                                                                                           |
| <code>cdo remapbil,r360x180 input.nc output_1deg.nc</code>        | Resamples data to a 1°x1° grid using bilinear interpolation.                                                                                                                                                                                                                                                            |
| <code>cdo -L monmean -selmon,5/9 input.nc output.nc</code>        | Chain operation, equivalent to:<br><code>cdo selmon,5/9 input.nc tmp.nc</code><br><code>cdo monmean tmp.nc output.nc</code><br><code>rm tmp.nc # delete the temporary file</code> <ul style="list-style-type: none"><li>• Faster execution</li><li>• Reduces I/O overhead</li><li>• No temporary files needed</li></ul> |

\*Information \*File Operation \*Selection \*Arithmetic & Statistics \*Regridding \*Chain Operation

### 3. Advantages of Using CDO Over Python/NCL

| Feature           | CDO                                | Python (e.g., xarray)             | NCL                                  |
|-------------------|------------------------------------|-----------------------------------|--------------------------------------|
| Ease of use       | ✓ One-liners for common tasks      | ✗ Often requires custom scripting | ✗ Steeper learning curve             |
| Speed             | ✓ Optimized C-based backend        | ✗ Slower for large files          | ✗ Slower I/O                         |
| Batch processing  | ✓ Ideal for command-line workflows | ✓ Possible, but more verbose      | ✓ but less flexible                  |
| Installation size | ✓ Lightweight                      | ✗ Heavier dependencies            | ✓ Lightweight                        |
| Learning curve    | ✓ Easy for non-programmers         | ✗ Requires programming basics     | ✗ Older syntax, harder for beginners |

### 4. When to Use CDO?

- You need **fast and simple operations** on large NetCDF/GRIB datasets.
- You're working in **shell environments** (e.g., HPC clusters).
- You want **quick diagnostics, filtering, or regridding** without writing code.